

Admin: Design Framework

Back to: [Documentation Index](#) **See also:** [Technical Reference](#) · [Security](#)

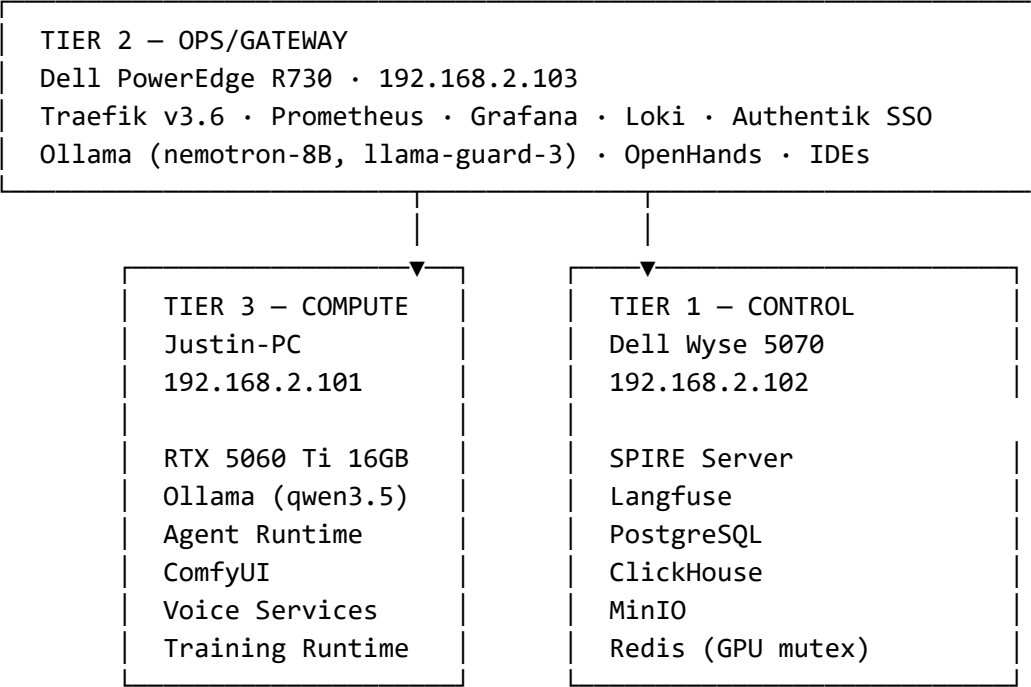
Version: 3.3 · **Status:** Production · **Last Updated:** 2026-03-23

1. System Philosophy

The Agentic Hive is built around three principles:

1. **Specialization over generalization:** Different tasks use purpose-built agents with the right model for the job. A routing model is fast and cheap; a coding model is deep and large. Neither substitutes for the other.
2. **Inference-time verification:** Output quality is enforced at runtime, not post-hoc. The MarsRL loop prevents bad output from ever reaching the user.
3. **Locality and privacy:** All inference runs on-premises. No external API calls. Training improves local models continuously from real usage data.

2. Three-Tier Hardware Architecture



Design Rationale

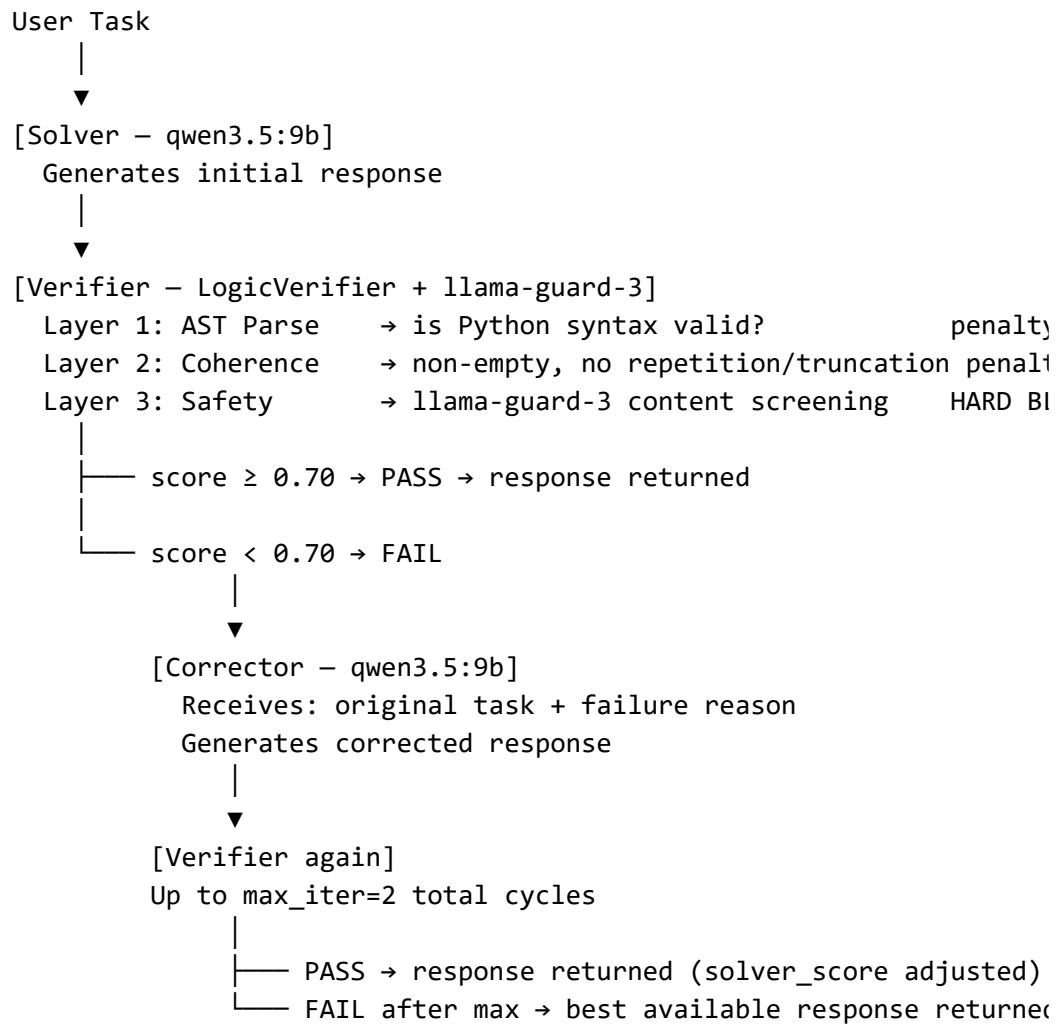
Tier	Why Separate?
Control (Wyse)	Low-power always-on node. Stores data, runs identity/auth. Isolated from compute to prevent data exfiltration if compute is compromised.
Compute (Justin-PC)	GPU workloads require isolation. Training, inference, and generation compete for 16GB VRAM — managed by Redis GPU mutex.
Ops/Gateway (R730)	Traffic routing and monitoring must be independent of compute. If Justin-PC is down, you can still see why via Grafana. R730's large RAM handles OpenHands, IDEs, and media stacks without GPU pressure.

3. MarsRL Inference-Time Loop

3.1 Overview

The MarsRL loop is the core quality enforcement mechanism. It implements the methodology from:

- **MarsRL** (November 2025): multi-agent process-reward signals for specialized agents
- **MiniMax Forge** (February 2026): process-level rewards, decoupled training/inference



3.2 Quality Scoring

Score	Meaning	Action
≥ 0.90	High quality	Tagged as training_candidate=1.0 in Langfuse
0.70–0.89	Acceptable	Returned to user as-is
< 0.60	Unacceptable	Corrector invoked
0.0	Safety block	Hard-blocked, Corrector not invoked

3.3 Solver Score Decay

solver_score penalizes multiple correction rounds:

- First-try pass: solver_score = 1.0
- After 1 correction: solver_score = 0.7
- After 2 corrections: solver_score = 0.4

This ensures that traces requiring heavy correction are weighted lower in the training dataset.

3.4 Non-Code Intent Paths

The Router dispatches non-code intents directly (no MarsRL loop):

Intent	Path
IMAGE / 3D	ComfyUI → cv2 check → Moondream VLM validation
IOT	Home Assistant API (safe_mode guard)
VOICE	Voice pipeline (Qwen-TTS → RVC)
RESEARCH	Direct Qwen 3.5 9B response (no verification)

4. Agent Architecture

4.1 Agent Catalog

Agent	Class	Model	Capabilities
Router	SemanticRouter	nemotron-orchestrator:8b	Intent classification, delegation
Solver / Architect	ArchitectAgent	qwen3.5:9b	Code generation, system design
Corrector	CorrectorAgent	qwen3.5:9b	Response repair given failure reason
Verifier	VerifierAgent	LogicVerifier + llama-guard-3:8b	Output validation
IoT Agent	IoTAgent	qwen3.5:9b	Home Assistant tool calls
BMO Agent	BMOAgent	qwen2.5:3b	Conversational voice (small model for VRAM efficiency)
Forge Agent	ForgeAgent	ComfyUI	Image/3D generation
Security Agent	SecurityAgent	Regex + llama-guard-3:8b	Threat screening

4.2 Health-Aware Routing

The NodeHealthMonitor checks both Ollama nodes every 30 seconds (cached TTL):

```
get_best_host_for_model(model_name):  
    1. Check Justin-PC VRAM headroom (via /api/ps)  
    2. If Justin-PC VRAM hot → route to R730  
    3. If R730 doesn't have the model → static fallback to Justin-PC
```

This ensures large-context tasks (qwen3.5:9b 256K) go to the node with available VRAM.

4.3 ExpertiseTemplate System

Every agent is backed by a versioned **ExpertiseTemplate** stored in `swarm.expertise_templates`:

```
ExpertiseTemplate {  
    id:            string           # e.g., "coding-expert"  
    intent:        string           # routes: CODE, RESEARCH, etc.  
    current_version: int            # auto-incremented on promotion  
    system_prompt: text             # the agent's persona prompt  
    capabilities:  string[]         # allowed tools: ["file_ops", "t  
    default_model: string           # e.g., "qwen3.5:9b"  
    security_level: string          # "standard" | "elevated" | "res  
}
```

Template evolution: When `avg_score` in `expertise_template_versions` exceeds the previous version by a configurable threshold, the version is auto-promoted and the template's `current_version` is bumped. This propagates to all new agent instances automatically.

5. Data Flows

5.1 Request Lifecycle

Browser/App

- R730 Traefik (L7 routing, Authentik forward-auth)
- Justin-PC:8008 (Agent Runtime FastAPI)
 - SPIFFE SVID validation (workload identity check)
 - JWT-ACE token issuance (capability gating)
 - Security Agent (llama-guard input screen)
 - SemanticRouter (intent classification)
 - Specialist Agent (MarsRL loop or direct)
 - Langfuse trace creation (process reward injection)
 - Performance history write (swarm.performance_history)
- Response streamed back to user

5.2 Training Data Flow

MarsRL execution

- Langfuse trace (spans: solver, verifier_round_N, corrector)
- Langfuse score (solver_score, verifier_round_N, final_quality, 1)

Periodically (or on-demand):

- export_traces.py
 - SELECT traces WHERE score 'training_candidate' = 1.0
 - Export to JSONL: execution_plane/training_data/traces.jsonl
 - Write swarm.training_runs record (run_type='export')

grp0_trainer.py (profile: training)

- QLoRA fine-tuning on JSONL (GRPO reward: correctness×0.5 + efficacy)
- GPU mutex: Redis lock (context="training") evicts Ollama + Comfy
- Output: execution_plane/training_output/grp0_<TIMESTAMP>/adapter

convert_gguf.py

- LoRA adapter → GGUF → ollama import
- Write swarm.model_versions record (status='candidate')
- A/B test auto-starts if template has an active version

5.3 Observability Data Flow

Agent Runtime → Prometheus metrics endpoint (:8008/metrics/)
↑ scraped by Prometheus (R730) every 15s

Agent Runtime → Langfuse API (Wyse 5070:3210)
→ traces, spans, scores written per request

Docker containers → Promtail (Docker socket)
→ Loki (R730:3100) indexed by {container, service, logstream}

cAdvisor (R730) + cAdvisor Proxy (Justin-PC:8081)
→ Prometheus ← container CPU/memory/network/disk

6. GPU Resource Management

VRAM Budget (Justin-PC — RTX 5060 Ti 16GB)

State	Allocations	Available
Inference only	Ollama (qwen3.5:9b ~8–9GB)	~6–7GB headroom
Inference + ComfyUI	Ollama + ComfyUI loaded	~2–3GB headroom
Training	Training runtime (~12.5GB)	~3GB

GPU Mutex

A Redis-based lock (`redis.lock("gpu_request", context="training")`) prevents concurrent GPU use during training:

- Training acquires the lock → sends EVICT signal to Ollama and ComfyUI
- Inference requests during training: mutex is **fail-open** (returns lock immediately) if Redis is unavailable — safe for single-user usage

Redis host: 192.168.2.102:6379 (Wyse 5070) **Password:** set in `execution_plane/.env` as `REDIS_PASSWORD`

⚠ Redis port 6379 must be exposed in `control_plane/docker-compose.yml`.
If the GPU mutex is not working, check this first.

7. Model Stack

Model	Size	VRAM	Location	Role
-------	------	------	----------	------

qwen3.5:9b	~5.5GB GGUF	~9GB loaded	Both nodes	Solver, Corrector, Architect
nemotron- orchestrator:8b	~5GB	~8GB	R730	Router/Orchestrator
llama-guard-3:8b	~5GB	~8GB	R730	Safety screening
qwen2.5:3b	~2GB	~3GB	Justin-PC	BMO conversational voice
Local fine-tuned models	varies	varies	Justin-PC	Promoted GRPO adapters